

ProgettoLC parte1 Gruppo 23-13 Relazione

strumento	versione
BNFC	2.9.5
Alex	3.5.1.0
Happy	1.20.1.1
GHC	9.4.8

Assunzioni sul linguaggio

Nello scrivere la grammatica abbiamo deciso di rendere possibile:

- la commutatività del prodotto, per cui è possibile scrivere, nel caso di `Tree Int Double`, `4 * 5.0` oppure `5.0 * 4`, che verranno entrambi interpretati come `Repeat 4 Leaf 5.0`.

Si nota che non si pongono problemi di associatività del prodotto, poiché un'espressione del tipo `4 * 4 * 5.0` non viene accettata in quanto il nodo figlio deve essere diverso dal nodo padre.

- l'introduzione di parentesi nelle sequenze di somme, sebbene questo non comporti una differenza in termini di semantica. Scrivere `4.0 + 5.0 + 3.0` o `(4.0 + (5.0)) + 3.0` produrrà comunque un nodo `Chain [Leaf 4.0, Leaf 5.0, Leaf 3.0]`. Un intero non può essere scritto fra parentesi perché non può essere usato autonomamente in sequenze di somme. Sempre per il vincolo della consegna, non è possibile scrivere espressioni del tipo `4 * (5 * 6.0)`.

Implementazione

Per produrre i parser e lexer abbiamo iniziato scrivendo due file BNFC, uno per le istanze di `Tree Int Double` e uno per quelle `Tree Int String` (i file `.cf` sono inclusi nella cartella). Successivamente abbiamo modificato i file Happy prodotti per

- appiattare le sequenze di `Chain`
- ridurre le due labels `Repeat1` e `Repeat2` corrispondenti alle due regole della moltiplicazione a un unico tipo `Repeat`
- togliere l'etichetta `IntVal` usata in BNFC
- utilizzare il tipo di dato astratto polimorfo della consegna.

La grammatica ha un'ambiguità data dalle produzioni:

- $E2 \rightarrow (E2)$
- $E2 \rightarrow E3$
- $E3 \rightarrow E4$
- $E4 \rightarrow (E4)$

Il parser contiene un conflitto shift/reduce allo stato 16 quando i due simboli in testa allo stack sono `(`, `E4` e il simbolo di lookahead è `)`. Non abbiamo modificato la scelta a favore dello shift perché entrambe le azioni fanno sì che si converga al simbolo `E2` in testa allo stack e senza una differenza in termini di abstract syntax tree prodotto.

Essendo i due file Alex molto simili li abbiamo fusi in un file unico, adatto sia alla versione *Double* che a quella *String*.

Punto b della consegna

L'intersezione fra la grammatica implementata nei parser e quella delle stringhe palindrome su $T = \{ a, b, *, +, (,) \}$, con `a` a rappresentare gli *Int* e `b` a rappresentare rispettivamente *String* o *Double* è generata dalla seguente grammatica context free:

- $S \rightarrow B \mid a * B * a \mid b$
- $B \rightarrow b + C + b \mid b + b$
- $C \rightarrow S$

La grammatica permette di scrivere sequenze di somme di elementi di tipo `b`, intercalate con moltiplicazioni per elementi di tipo `a`.

Esempi di stringhe prodotte dalla grammatica:

```
b
b + b
...
b + b + b + b
a * b + b + b * a
b + a * b + b * a + b
b + b + a * b + b * a + b + b
a * b + b + a * b + b * a + b + b * a
a * b + a * b + a * b + b * a + b * a + b * a
a * b + b + a * b + a * b + b * a + b * a + b + b * a
```

Esempi di stringhe non producibili:

```
a * b * a
a * a
b * b
a * a * b + b * a * a
```

Test e demo

Per compilare i sorgenti è sufficiente lanciare il comando `make`.

Per valutare i punti **c**, **d**, **e** della consegna, una volta lanciato `make`, si può utilizzare l'eseguibile `Evaluate` in questo modo (per esempio con due file):

```
./Evaluate ./tests/main-tests1.txt ./tests/main-tests2.txt
```

Per lanciare i parser con dei file in input si possono usare i due eseguibili `Double` e `String` con argomento un file contenente le stringhe per cui vuole fare il parse.

Per lanciare automaticamente una demo, si può usare il comando `make demo`.